Software Design Document

Department Course Timetable Management System | SENG 271

Field	Value
Version	SDD v2 - corrected Code v1 design
Prepared	10.11.2026
Team	Team 19
Members	Eren KARAKAŞ (@ErennKarakass); Semih Mertcan İÇYER (@semihicyer)
Repository	https://github.com/ErennKarakass/Daviser
Status	Skeleton implementation; full scheduling design is planned, not implemented.

Revision History

Version	Date	Change
Earlier v2 draft	10.11.2026	Received team draft; earlier claimed history not independently verified.
v2 corrected	10.11.2026	Aligned all 13 classes to current source; corrected flow branches and loops; simplified timetable storage to one canonical list; specified pending features.

1. Introduction

This design implements the corrected SRS v2. Code v1 contains model constructors, basic accessors, course-to-session creation, interval bounds/overlap and a runnable menu. All scheduling, file loading, reports and manual-edit operations remain explicit stubs.

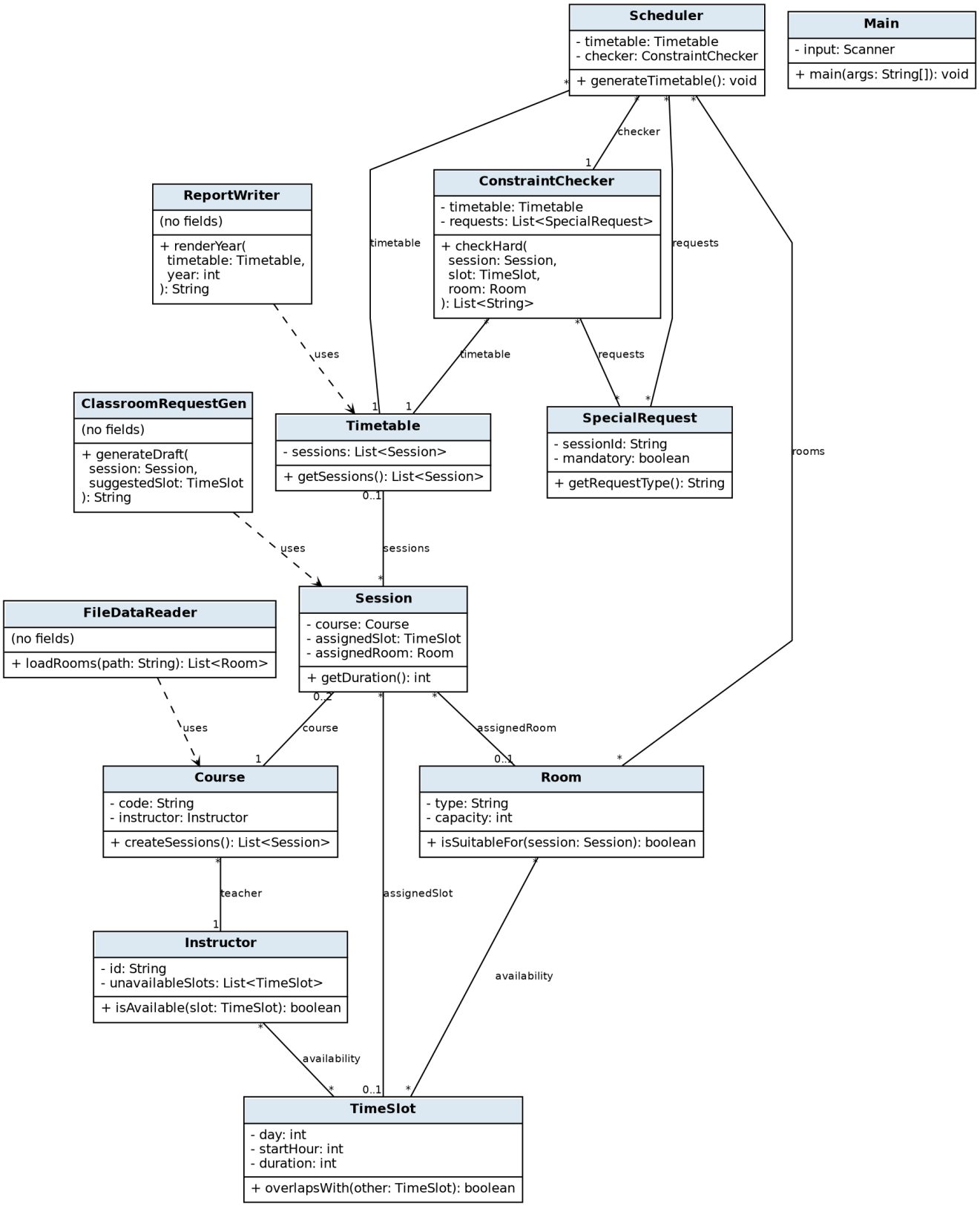
2. System Overview

Part	Classes	Current state
Interface	Main	Implemented menu only
Input	FileDataReader	Declared, not implemented
Domain	Course, Session, Instructor, Room, TimeSlot, SpecialRequest	Constructors/basic model implemented; availability/suitability stubs
Scheduling	Timetable, Scheduler, ConstraintChecker	Constructors/accessors; operations are stubs
Output	ClassroomRequestGen, ReportWriter	Declared, not implemented

Menu options do not call unfinished operations. Stubs throw UnsupportedOperationException when invoked directly, rather than returning a misleading successful result.

3. UML Class Diagram

Source: docs/design/class-diagram.dot. The overview shows all classes, selected important members, associations and multiplicities. Dashed arrows denote method-signature dependencies. Main currently owns only its Scanner; planned feature dispatch dependencies are not claimed as implemented. Full signatures are listed in the companion member inventory.



The following class inventory is generated from the same model as the Java source; constructor and method declarations can be checked directly against src/. The diagram source is editable Graphviz DOT.

4. Class Responsibilities

Class	Responsibility
TimeSlot	Stores one consecutive teaching interval; validates bounds and checks overlap.
Instructor	Stores instructor identity, unavailability and preferences; availability logic is planned.
Room	Stores room identity, type, capacity and availability; suitability checks are planned.
Course	Stores a course and creates a fresh theory/laboratory block list when requested.
Session	Stores one teaching block and its optional assignment; only the planned validated Timetable operations may assign it in the user workflow.
SpecialRequest	Represents a block-specific scheduling request and its mandatory/preferred classification.
FileDataReader	Will parse and validate four input files in dependency order; all reader methods are stubs.
Timetable	Will hold the canonical session list and manage validated assignments, moves and removals.
ConstraintChecker	Will return all relevant hard-rule failures or soft warnings for a proposed assignment.
Scheduler	Will run first-fit scheduling, retain failure reasons and collect preference warnings.
ClassroomRequestGen	Will create and save text-only room requests for confirmed room-shortage cases.
ReportWriter	Will format timetables, scheduled lists, failure reports and output files.
Main	Runs the console menu, handles invalid input and exits cleanly; feature options are visibly unfinished.

5. Design Decisions

Question / alternatives	Decision and reason
Session representation: embed hours vs separate blocks	Separate Session objects, with Course reference, T/L type and duration. Generate once on data load; regeneration reuses those objects to avoid duplicates.
Time representation: hour cell vs interval	TimeSlot is a consecutive interval (day 1-5, start hour, duration). The constructor enforces working bounds; overlapsWith uses half-open interval overlap.
Timetable: grid vs list vs both	One canonical List<Session>, with optional assignment fields. Simpler consistency than parallel grid/list storage; rendering derives the year view. Every clash check scans existing assignments.
Availability: flags vs intervals	List<TimeSlot> on Instructor and Room. Check overlap across the whole candidate block.
Special requests	SpecialRequest targets a session ID, so theory and lab cannot ambiguously share a fixed request. Store request type, encoded value and mandatory flag.
Hard checks	ConstraintChecker.checkHard returns violation codes/messages; assign and modify reuse it. Mandatory specific-room restrictions are checked in addition to HC-01..09.
Input and errors	FileDataReader parses into temporary collections; commit the dataset only after all required files open. Skip malformed rows with context; validate references in dependency order.
Collections	ArrayList for ordered objects and intervals, Map for instructor lookup, LinkedHashMap for ordered session failures. Use unmodifiable views from reporting accessors.

6.1 Scheduling Strategy

1. Process mandatory fixed-time sessions first, ordered by session ID. For others: theory before laboratory, year ascending, duration descending, student count descending, then session ID. This deterministic first-fit rule is intentionally simple.

2. Generate consecutive candidate intervals that finish by 17:00. For a 3-hour block, the last start is 14:00, not 16:00. A mandatory fixed-time block gets only its declared interval. Never retry a failed mandatory fixed block at a different time.

3. Rank valid candidates: preferred fixed time, theory-before-lab, actual instructor preference, fewer year-group gaps, then day/start. Preferred room first; then capacity ascending and room ID. Mandatory specific-room requests exclude all other rooms. Preference consideration is planned for v5.

4. Check every hard constraint plus mandatory room requests before assignment. If a candidate fails, try another eligible room; after rooms are exhausted, try the next allowed interval. A passing candidate is assigned and its soft warnings recorded.

5. If all candidates fail, classify NO ROOM only when some allowed interval satisfies non-room constraints but no permitted room passes room type/capacity/availability/occupancy. Otherwise use CONSTRAINT. Report multiple causes where applicable. This prevents requesting rooms for an instructor-time conflict.

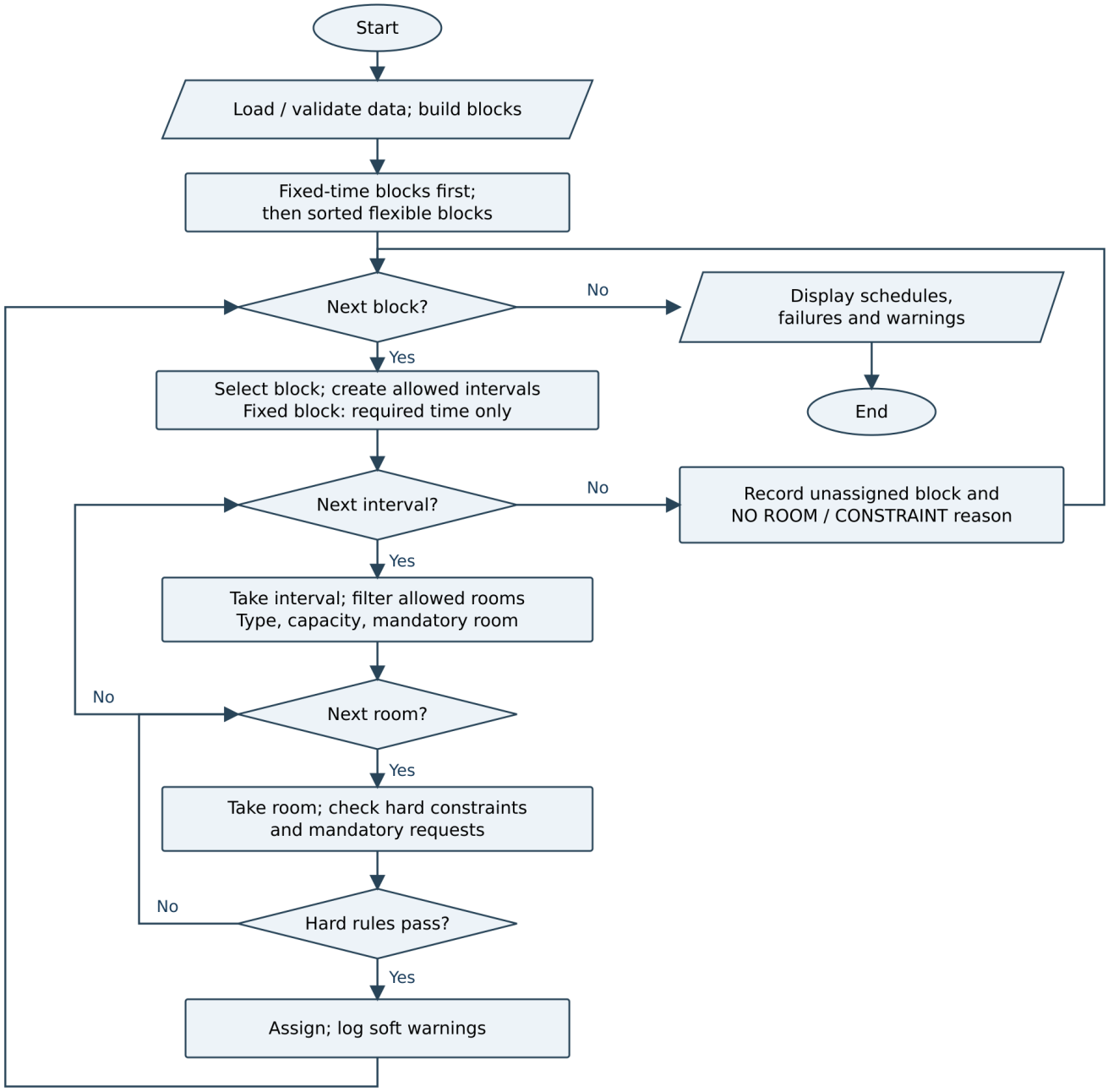
6. Continue until each session has been processed once. Report unassigned blocks and warnings. Do not infer mathematical infeasibility from greedy failure.

6.5 Known Limitations

No backtracking or global optimization. Earlier choices can block later sessions although another complete schedule exists. The algorithm may accept a soft-preference violation. Planned parsers do not support quoted delimiters. Code v1 cannot yet load files or generate schedules.

6.2 Automatic Scheduling Flowchart

Planning artifact for v3-v5. Each decision has Yes/No exits; rejected candidates return to room/slot search, successful assignments return to session selection. Source: scheduling-flowchart.svg (DOT topology also supplied).



6.3 Pseudocode (planned)

```
load_and_validate_data()
create_sessions_once()
clear_assignments_failures_and_warnings()
for session in ordered_sessions:
    placed = false
    slots = allowed_intervals(session)  # fixed: one interval
    for slot in ranked(slots):
        for room in ranked(eligible_rooms(session)):
            if checkHard(session, slot, room).isEmpty():
                assign(session, slot, room)
                record(checkSoft(session, slot, room))
                placed = true
                break
        if placed: break
    if not placed:
        record_failure(session, classify_using_room_and_nonroom_checks())
display_timetables_and_problem_report()
```

6.4 Hard-Constraint Implementation Map

ID	Method / current status	Planned tests
HC-01	ConstraintChecker.checkHard; planned v3-v4.	TC-02
HC-02	ConstraintChecker.checkHard; planned v3-v4.	TC-03
HC-03	ConstraintChecker.checkHard; planned v3-v4.	TC-04
HC-04	TimeSlot constructor checks interval bounds in v1; ConstraintChecker.checkHard will integrate HC-04 in v3.	TC-16
HC-05	ConstraintChecker.checkHard; planned v3-v4. Delegates to Room.isSuitableFor.	TC-05
HC-06	ConstraintChecker.checkHard; planned v3-v4. Delegates to Room.isSuitableFor.	TC-05, TC-14
HC-07	ConstraintChecker.checkHard; planned v3-v4. Delegates to Instructor.isAvailable.	TC-06
HC-08	ConstraintChecker.checkHard; planned v3-v4.	TC-07
HC-09	ConstraintChecker.checkHard; planned v3-v4. Delegates to Room.isAvailable.	TC-17

7. Conflict Detection and Handling

Problem	Planned action / user feedback
Instructor or year-group clash	Reject candidate; try another allowed interval. Identify HC-01, HC-03 or HC-07.
Room clash, unavailable room, insufficient capacity or wrong type	Try remaining rooms, then other allowed intervals. Do not stop at the first unsuitable room.
Fixed-time request conflicts with a hard rule	Try eligible rooms at that same fixed interval only. Leave unassigned if none work; explain the conflicting rules.
No valid assignment	Classify using separate room/non-room checks; report reason. Never force placement.
Soft violation	Accept hard-valid placement; record warning. Manual moves show relevant warnings.
Manual move	Evaluate replacement while excluding the session's own old placement. Only replace on success; preserve original on rejection.
Regeneration	Clear all assignment states/failures/warnings; keep loaded entities and recompute.

8. User Interface Design

Option	Planned destination	Requirement
1 Load data	FileDataReader load methods; Course.createSessions; Timetable.setSessions	FR-01..05, 24, 25
2 Generate	Scheduler.generateTimetable	FR-06..10, 22, 23
3 View year	ReportWriter.renderYear	FR-11
4 View all	ReportWriter.renderScheduled	FR-12
5 Problems	ReportWriter.renderProblems	FR-13, 14
6 Modify/remove	Timetable.modifyAssignment/removeAssignment	FR-15..17
7 Email draft	ClassroomRequestGen.generateDraft/saveToFile	FR-18..20
8 Save output	ReportWriter.saveAll	FR-21
0 Exit	Main.main	NFR-02

Code v1: Main.handleChoice displays a truthful not-implemented message for options 1-8. No data processing or empty fake schedule is presented.

9. Requirements Traceability (1/2)

Requirement	Declared method	Test / target
FR-01	FileDataReader.loadCourses	TC-12 / v2
FR-02	FileDataReader methods	TC-12 / v2
FR-03	FileDataReader.loadInstructors	TC-12 / v2
FR-04	FileDataReader.loadRooms	TC-12 / v2
FR-05	FileDataReader.loadRequests	TC-07, TC-12 / v2
FR-06	Scheduler.generateTimetable	TC-01, TC-14 / v3-v4
FR-07	Scheduler.generateTimetable	TC-07 / v3
FR-08	Scheduler.generateTimetable	TC-09 / v3
FR-09	ConstraintChecker.checkHard	TC-02..07, TC-14, TC-17 / v3-v4
FR-10	Scheduler.generateTimetable	TC-10 / v3
FR-11	ReportWriter.renderYear	TC-01 / v4
FR-12	ReportWriter.renderScheduled	TC-01 / v4
FR-13	ReportWriter.renderProblems	TC-10 / v4

All rows here describe planned functional behavior. Declaring a method is not implementing its requirement. HC coverage is in §6.4; the next page covers remaining FRs and NFRs.

9. Requirements Traceability (2/2)

Requirement	Declared method	Test / target
FR-14	ConstraintChecker.checkSoft; ReportWriter.renderProblems	TC-08, TC-19 / v5
FR-15	Timetable.modifyAssignment	TC-13 / v5
FR-16	Timetable.modifyAssignment; ConstraintChecker.checkHard	TC-13 / v5
FR-17	Timetable.removeAssignment	TC-13 / v5
FR-18	Scheduler.generateTimetable	TC-05, TC-10, TC-17 / v4
FR-19	ClassroomRequestGen.generateDraft	TC-11 / v6
FR-20	ClassroomRequestGen.saveToFile	TC-11 / v6
FR-21	ReportWriter.saveAll	TC-18 / v6
FR-22	Scheduler.generateTimetable; ConstraintChecker.checkSoft	TC-08, TC-19 / v5
FR-23	Scheduler.generateTimetable	TC-20 / v4
FR-24	FileDataReader methods; Main.handleChoice	TC-12 / v2
FR-25	FileDataReader methods; Course.createSessions; Timetable.setSessions	TC-01, TC-12 / v2
NFR-01	Main.main now; FileDataReader later	TC-15 (menu); TC-12 (v2)
NFR-02	Main.main / handleChoice	TC-15 / v1
NFR-03	Scheduler.generateTimetable	TC-21 / v4
NFR-04	All classes / UML review	TC-22 / each release

10. Changes from Supplied v2 Draft

Preserved 13 class responsibilities but standardized ClassroomRequestGen; replaced incomplete signatures with exact source declarations; removed unsupported implementation claims. Timetable uses one list rather than synchronized arrays and a list. Added correct flow branches/loops and fixed-time protection. Pseudocode is optional, not a Checkpoint 1 requirement. No claim is made that historical v1 files or releases exist.

AI-Use Declaration

Tool	Used for	Verification / remaining review
ChatGPT (Codex)	Requirements revision, design and diagram generation, Java skeleton, documentation and build checks.	Assistant checked source/diagram alignment and compiled/exercised the skeleton in its environment. Both team members must still review, run and explain the result; their approval is not claimed.